Algerian Highway Toll & Speed Enforcement System

طريق شرق غرب — Tariq Sharq-Gharb

Project Architecture & Technical Report

Abdelkader Boukert • Software Engineer & DevOps Engineer

1. Project Overview

The Algerian East-West Highway (Tariq Sharq-Gharb) is a 1,216 km national infrastructure project connecting the Moroccan border to the Tunisian border. This system provides a complete software platform to manage toll collection, vehicle tracking, speed enforcement, and real-time checkpoint coordination across the highway.

The platform is built as a distributed microservices architecture where each service handles one specific domain, communicates through Kafka event streams, and is deployed independently on Kubernetes. The system is designed to handle thousands of vehicle events per hour across dozens of checkpoints simultaneously.

2. Problem Statement

The highway requires a system that can:

- Allow drivers to pay a toll at the entry checkpoint based on their destination
- Validate at every intermediate checkpoint that the driver has a valid paid pass
- Detect vehicles that pass without paying and alert all downstream checkpoints in real time
- Detect vehicles exceeding the speed limit (120 km/h) using camera pairs at each gate
- Identify vehicle owners by plate number for enforcement purposes
- Automatically retrain the plate detection model as new data is collected
- Give highway operators a live dashboard of all activity across all checkpoints

3. System Architecture

The system is divided into three layers: the Checkpoint Layer (what runs at each physical toll gate), the Central System (the core microservices), and the DevOps Layer (infrastructure and deployment).

Layer	Components
Checkpoint Layer	LPR Service, Speed Detection Service, Camera feeds (RTSP)
Central System	Payment, Permission, Alert, Notification, Owner Lookup, Model Training, Model Serving
Frontend	Next.js Operator Dashboard
Messaging	Apache Kafka — event stream between all services
Storage	PostgreSQL (3 databases), Redis (speed state), AWS S3 (models, images)
DevOps	Docker, Kubernetes (EKS), Terraform, ArgoCD, GitHub Actions, Prometheus, Grafana

4. Microservices

4.1 Payment Service

Property	Value
Language	Python 3.11 + FastAPI
Database	PostgreSQL — toll_db
Port	8001

The entry point for every toll transaction. When a driver arrives at a checkpoint, they provide their plate number and destination. This service calculates the fare, creates a digital toll pass linked to the plate number, and returns a confirmation token.

Key Tables

- checkpoints — all highway toll gates with km markers
- toll_passes — active and historical passes (plate, entry, exit, amount, status, expiry)
- fare_matrix — price between any two checkpoints

Fare Logic

- Base rate: 2 DZD/km for cars, 4 DZD/km for trucks
- Minimum fare: 50 DZD
- Pass valid for 24 hours from payment

API Endpoints

```
POST /api/v1/passes/create          — create new toll pass
GET  /api/v1/passes/{plate}/active — check active pass for plate
GET  /api/v1/fare                   — calculate fare between checkpoints
```

4.2 Permission Service

Property	Value
Language	Python 3.11 + FastAPI
Database	PostgreSQL — toll_db (read-only)
Port	8002

Called in real time at every checkpoint when a plate is detected. Answers one question: does this car have a valid pass to pass through this specific checkpoint? Checks that the car's pass is active, not expired, and that the current checkpoint falls within the paid route segment.

Checkpoint Range Logic

A car is allowed if the current checkpoint km marker falls between the entry and exit checkpoint km markers of its active pass. Example: a car paid from Blida (km 280) to Setif (km 580) is allowed at any checkpoint between those two km markers.

Kafka Topics Produced

- toll.unpaid — when a car has no valid pass (consumed by Alert Service)

4.3 Alert Service

Property	Value
Language	Python 3.11 + FastAPI
Database	PostgreSQL — alert_db
Port	8003

The enforcement brain of the system. Receives events for unpaid tolls and speed violations, creates alert records, and broadcasts them to all downstream checkpoints in real time.

Alert Types & Expiry Rules

Alert Type	Trigger	Expires	Record
unpaid_toll	Car passed without paying	24 hours	Permanent
speed_violation	Car exceeded 120 km/h	Never	Permanent

Kafka Topics

- Consumed: toll.unpaid, speed.violation
- Produced: alert.broadcast — sent to all checkpoints on new alert

4.4 Notification Service

Property	Value
Language	Python 3.11 + FastAPI + WebSocket
Port	8004

Delivers real-time notifications to drivers (SMS via Twilio) and to the operator dashboard (WebSocket push). Drivers are notified when their vehicle is flagged. Operators see live alerts without refreshing.

4.5 Owner Lookup Service

Property	Value
Language	Python 3.11 + FastAPI
Database	PostgreSQL — owner_db
Port	8005

Returns the registered owner of a vehicle given its plate number. In production this queries the Algerian CNRC/DGSN vehicle registry. For the portfolio version the database is seeded with 500 realistic fake Algerian vehicle owners (Faker library).

Algerian Plate Format

Standard format: NNNNN NNN NN — example: 12345 124 16. All plate inputs are validated against this format.

4.6 LPR Service (License Plate Recognition)

Property	Value
Language	Python 3.11 + FastAPI + OpenCV + YOLOv8
Port	8006

The eyes of the system. Receives camera frames from checkpoint cameras, runs the 3-stage cascaded detection pipeline to extract the plate number, and publishes the result to Kafka.

Stage 1 — Vehicle Detector (Wide Net)

- Model: YOLOv8n pretrained on COCO dataset
- Input: Full 1080p or 4K frame from checkpoint camera
- Output: Bounding box crop of the detected vehicle
- Goal: Reduce image by ~80% before Stage 2. Can run at the edge on the camera gateway to save bandwidth
- Detects: car, truck, bus, motorcycle

Stage 2 — Plate Localizer (Zoom)

- Model: YOLOv8n custom trained — 433 real images + CCPD augmented with Algerian plate color shifts (yellow/white)
- Input: Vehicle crop from Stage 1
- Output: Perspective-corrected license plate crop
- Goal: Find the plate and flatten it using cv2.getPerspectiveTransform to correct for angle

Stage 3 — Character Recognizer (Reader)

- Model: YOLOv8 trained to detect individual digits as objects
- Training Data: 100,000 synthetic Algerian plates generated with Pillow + albumentations
- Synthetic data includes: font variation, rotation ±5°, brightness/contrast, motion blur, dirt overlays, partial shadows
- Post-processing: sort detected digits by x-coordinate, validate against Algerian plate format

Kafka Topics Produced

- plate.detected — plate number, checkpoint ID, camera ID, confidence, timestamp

4.7 Speed Detection Service

Property	Value
Language	Python 3.11 + FastAPI
Databases	Redis (temp state) + PostgreSQL — alert_db
Port	8007

Calculates vehicle speed using two cameras placed 50 meters apart at each checkpoint. Camera 1 records the entry timestamp, Camera 2 records the exit timestamp. Speed is calculated from the known distance and elapsed time.

Speed Calculation

```
Speed (km/h) = (Distance in meters / Time in seconds) × 3.6
```

Redis State

Camera 1 detection stores the timestamp in Redis with a 30-second TTL. If Camera 2 does not detect the same plate within 30 seconds the Redis key expires automatically with no violation recorded.

Kafka Topics Produced

- speed.violation — when speed exceeds 120 km/h

4.8 Model Training Service

Property	Value
Language	Python 3.11 + MLflow + YOLOv8
Trigger	GitHub Actions CRON every Sunday 2am, or manual
Port	8008

Retrains the YOLOv8 plate detection model on a schedule. Downloads the latest labeled dataset from S3, runs training, logs metrics to MLflow, and uploads the new model to S3 if performance improved. Calls Model Serving Service to hot-reload the new weights.

Training Pipeline

- Download latest labeled dataset from S3
- Split 80% train / 10% val / 10% test
- Start MLflow experiment run
- Train YOLOv8n for 50 epochs
- Log mAP50, precision, recall to MLflow
- If mAP50 improved: upload model to S3, call Model Serving /reload

4.9 Model Serving Service (Rust)

Property	Value
Language	Rust + Actix-web + ONNX Runtime
Port	8009

High-throughput inference API written in Rust for maximum performance. At highway scale with dozens of simultaneous camera feeds, inference latency matters. Serves the YOLOv8 model exported to ONNX format. Supports hot model reload without restarting.

Hot Reload

The ONNX model is stored behind an Arc<RwLock<Session>>. On a /reload request the service acquires a write lock, loads new weights from S3, and releases the lock. Ongoing inference requests are unaffected during the swap.

5. Frontend Dashboard

Property	Value
Framework	Next.js 14 (App Router, TypeScript)
Styling	Tailwind CSS + shadcn/ui
Charts	Recharts
Real-time	Socket.io-client (WebSocket)
Port	3000

Pages

- /dashboard — live overview: alerts, detections, revenue, violations
- /checkpoints — all checkpoints with live green/red status
- /checkpoints/[id] — single checkpoint: camera feed, recent plates
- /alerts — full alert table with filters
- /violations — speed violations table with filters
- /vehicles/[plate] — full history for one plate
- /analytics — revenue charts, traffic volume, violations by checkpoint

6. Technology Stack

Layer	Technology
API Microservices	Python 3.11 + FastAPI + Uvicorn
Model Serving	Rust + Actix-web + ONNX Runtime
Frontend	Next.js 14 + TypeScript + Tailwind + shadcn/ui
Message Broker	Apache Kafka + Zookeeper
Cache / Speed State	Redis 7
Databases	PostgreSQL 15
Plate Detection	YOLOv8n (Ultralytics) + OpenCV
Synthetic Data	Pillow + albumentations
ML Experiment Tracking	MLflow
Model Storage	AWS S3
Containerization	Docker (multi-stage builds)
Orchestration	Kubernetes (AWS EKS)
GitOps / CD	ArgoCD
CI/CD	GitHub Actions
Infrastructure as Code	Terraform
Monitoring	Prometheus + Grafana
Security Scanning	Trivy + Semgrep + TruffleHog + pip-audit

7. Data Flow

Happy Path — Car has valid pass

- Car enters checkpoint
- Camera → LPR Service detects plate
- Permission Service checks toll_db → pass is active and covers this checkpoint
- Gate opens
- Speed Service calculates speed → within limit → no violation

Unpaid Toll Path

- Car enters checkpoint
- Permission Service finds no active pass
- Publishes to Kafka: toll.unpaid
- Alert Service creates 24h alert and broadcasts to all checkpoints
- Notification Service sends SMS to driver
- If car detected again within 24h → timer resets
- If not detected in 24h → alert expires, record stays in DB permanently

Speed Violation Path

- Car passes Camera 1 → timestamp stored in Redis (30s TTL)
- Car passes Camera 2 → speed calculated: e.g. 147 km/h
- Speed Service publishes to Kafka: speed.violation
- Alert Service creates permanent violation record
- Notification Service notifies driver and operator

8. Build Order

Build in this exact order. Do not skip phases.

Phase	Task
Phase 1	Monorepo scaffold + docker-compose
Phase 2.1	Payment Service
Phase 2.2	Permission Service
Phase 2.3	Alert Service + Kafka
Phase 2.4	Notification Service + WebSocket
Phase 2.5	Owner Lookup Service + data seeder
Phase 3.1	LPR Service — 3-stage pipeline
Phase 3.2	Speed Detection Service — Redis state machine
Phase 4.1	Model Training Service — MLflow
Phase 4.2	Model Serving Service — Rust
Phase 5.1	Next.js Dashboard
Phase 6.1	All Dockerfiles (multi-stage)
Phase 6.2	GitHub Actions CI/CD

Phase 6.3	Helm charts
Phase 6.4	Terraform (AWS)
Phase 6.5	ArgoCD manifests
Phase 7.1	Prometheus + Grafana dashboards

Abdelkader Boukert

Software Engineer & DevOps Engineer

github.com/abdelkderboukert • abdelkaderboukart@gmail.com